Data Processing and Reporting - full solution notes

This PDF is the complete write-up of my solution. It is meant to stand alone: setup, loading, both reports, performance, outputs, and the reasons behind the main choices. LOGICAL_STEPS.txt and PERFORMANCE_REPORT.txt are the shorter required TXT files. Everything important from those files is also written here.

1. What I was asked to do

- Load accounts, daily updates and monthly snapshots into a database (SQLite).
- After load, process using DB records only, not the CSV files.
- Find accounts with activity on or after 2025-01-01, join account details, export CSV.
- Find the most recent change for accounts that as of 27 Nov 2025 are in COLLECTIONS or LEGAL.
- Measure reading performance for daily statuses + account details for accounts that have been in those queues; improve the query; compare before and after.

Stack: Python, SQLite, Polars (Polars only to read CSVs). Database file: db/reporting.db (local, gitignored).

2. How to run

Setup (creates .venv and installs requirements.txt):

```
make setup
```

Full run:

```
make run
```

Or step by step:

```
.venv/bin/python main.py load
.venv/bin/python main.py process
.venv/bin/python main.py perf
```

- load: rebuild DB from data/, fix datetimes, create indexes
- process: build both report tables and write both CSVs
- perf: time slow vs faster query, write PERFORMANCE_REPORT.txt

Also available: make load, make process, make perf, make clean.

3. Project layout

```
main.py              CLI entry (load / process / perf / all)
src/config.py        paths, cutoffs, target queues
src/db.py            schema, connect, indexes
src/load.py          CSV load + datetime cleanup
src/transform.py     activity + COLLECTIONS/LEGAL reports
src/performance.py   timing + PERFORMANCE_REPORT.txt
data/                source CSVs
outputs/             report CSVs + performance TXT
LOGICAL_STEPS.txt    short logical notes (required)
SOLUTION_NOTES.pdf   this document
README.md            quick start
```

4. Database schema

Source tables:

- accounts(id, name, address) -- id comes from account_id in accounts.csv
- daily_status(account, queue, status, changed_datetime, source_file), UNIQUE(account, changed_datetime)
- monthly_status(account, queue, status, year_month, month_num, source_file), UNIQUE(account, year_month)

Result tables:

- account_activity_latest -- rows for the activity CSV
- collections_legal_asof -- rows for the COLLECTIONS/LEGAL CSV

Indexes created after load:

- idx_daily_account_dt on daily_status(account, changed_datetime)
- idx_daily_dt on daily_status(changed_datetime)

- idx_daily_queue on daily_status(queue)
- idx_monthly_ym on monthly_status(account, year_month)

5. Loading (task 1)

This section matches LOGICAL_STEPS.txt, with more detail.

File to table mapping

accounts.csv goes into accounts. I rename account_id to id.

daily_YYYYMMDD.csv files go into daily_status.

monthly_YYYYMM.csv files go into monthly_status. The month column in those files is only 01, 02, etc, so I take year and month from the filename and store year_month as YYYY-MM (and month_num as the integer month).

Datetime cleanup

Most changed_datetime values already look like 2025-11-26 01:31:07. I normalize everything to YYYY-MM-DD HH:MM:SS so comparisons are reliable.

- Short EU style example: 10-12-24 00:36 is treated as DD-MM-YY and becomes 2024-12-10 00:36:00
- Excel serial example: 45667.06037 is converted using Excel epoch 1899-12-30; fractional seconds are dropped

On this dataset I fixed 27 odd values: 13 EU-style on 2024-12-10 and 14 Excel-style on 2025-01-10.

After a successful load: accounts = 501, daily_status = 3625, monthly_status = 5476.

Checks after load

- Refuse empty source tables
- Check that daily.account values exist in accounts
- Create indexes and run ANALYZE

From this point on, reports and performance work only talk to SQLite.

6. Activity since 2025-01-01 (tasks 2 and 3)

Source: daily_status only. An account is included if it has any change with changed_datetime >= 2025-01-01 00:00:00. For each included account I keep the newest change in that window, then join accounts for name and address.

How I select the latest row

I use ROW_NUMBER() OVER (PARTITION BY account ORDER BY changed_datetime DESC) and keep rn = 1. That keeps queue and status on the same row as the timestamp. Using only MAX(changed_datetime) would give the time but need another join to bring queue/status back.

Why not monthly here

Monthly data is an end-of-month snapshot, not a log of activity. The question asks about activity (queue or status change), so daily is the correct source.

Output

Stored in account_activity_latest and written to outputs/accounts_activity_since_2025.csv

Columns: account_id, name, address, latest_update_datetime, queue, status

Row count on this dataset: 498

7. COLLECTIONS / LEGAL as of 27 Nov 2025 (task 4)

As of Nov 27th 2025 means: take the last daily update strictly before 2025-11-27 00:00:00. The daily files stop on 2025-11-26, so that is the last real day we have.

Same latest-row pattern as above (ROW_NUMBER per account where changed_datetime < as-of), then keep rows whose queue is COLLECTIONS or LEGAL. The CSV uses the same columns as the activity file.

Daily vs monthly cross-check

I also counted COLLECTIONS/LEGAL in monthly_status for year_month = 2025-11. On this run: daily as-of = 141 accounts, monthly = 145. Close but not identical. I keep daily as the answer because it is the change log. Monthly is a snapshot and can differ after moves inside the month.

Monthly fallback edge case

Account 37924 (Laura Davis) had no daily_status row before the as-of cut-off, but appeared as LEGAL / HOLD FO in the November monthly snapshot. For that account I used the monthly row and set latest_change_datetime to 2025-11-26 23:59:59 (one second before as-of). That keeps it in the as-of set with a timestamp that is clearly before Nov 27, without inventing a fake real daily event.

Output

Stored in collections_legal_asof and written to outputs/latest_change_collections_legal.csv

Columns: account_id, name, address, latest_update_datetime, queue, status

Row count on this dataset: 141

8. Performance (task 5)

This section includes what is in PERFORMANCE_REPORT.txt, plus explanation.

Query under test: return daily status rows with account name/address for accounts that have ever been in COLLECTIONS or LEGAL.

Slow SQL (starting point)

```
SELECT d.*, a.name, a.address
FROM daily_status d
JOIN accounts a ON a.id = d.account
WHERE d.account IN (
    SELECT DISTINCT account
    FROM daily_status
    WHERE queue IN (?, ?)
)
```

For the slow timing I drop my secondary indexes and use a small SQLite cache (PRAGMA cache_size = -2000).

Faster SQL

```
SELECT d.account, d.queue, d.status, d.changed_datetime, d.source_file,
       a.name, a.address
FROM daily_status d
JOIN accounts a ON a.id = d.account
JOIN good_accounts g ON g.account = d.account
```

Changes I made

- Logical: build good_accounts once (DISTINCT account where queue in COLLECTIONS or LEGAL), then join to it
- Logical: select only needed columns instead of d.*
- Operational: recreate indexes and run ANALYZE
- Operational: raise cache_size to about -20000

Why good_accounts helps

The slow query rediscovers qualifying accounts with DISTINCT over daily_status on every run. good_accounts is a small table of those account ids (444 on this dataset). Joining to it avoids repeating that discovery work.

Measured numbers (median of 10 runs on my machine)

- Slow: 4.38 ms median, 3321 rows
- Faster: 3.52 ms median, 3321 rows
- About 1.24x faster
- Same row count both times

The speedup is modest because the dataset is small. I care more that the approach is correct and checked than about a huge factor.

EXPLAIN QUERY PLAN

Slow plan:

```
SEARCH a USING INTEGER PRIMARY KEY (rowid=?)
LIST SUBQUERY 1
```

```
SCAN daily_status USING INDEX sqlite_autoindex_daily_status_1
REUSE LIST SUBQUERY 1
SEARCH d USING INDEX sqlite_autoindex_daily_status_1 (account=?)
```

Faster plan:

```
SCAN a
SEARCH g USING INTEGER PRIMARY KEY (rowid=?)
SEARCH d USING INDEX idx_daily_account_dt (account=?)
```

Even after I drop my secondary indexes, SQLite still keeps the UNIQUE autoindex on (account, changed_datetime) from the table definition. So the slow version is not a pure heap scan. I still compare with/without my added indexes and with the rewritten SQL.

9. Deliverables checklist

- outputs/accounts_activity_since_2025.csv -- tasks 2-3 (498 rows)
- outputs/latest_change_collections_legal.csv -- task 4 (141 rows)
- outputs/PERFORMANCE_REPORT.txt -- task 5 before/after
- LOGICAL_STEPS.txt -- short logical steps
- SOLUTION_NOTES.pdf -- this full write-up
- Code + README + Makefile to reproduce the run

10. Interview-style short answers

Why normalize datetimes on load?

Latest-row logic and date filters only work if every value uses one comparable format. EU short dates and Excel serials would compare wrong if left as-is.

Why ROW_NUMBER instead of MAX(datetime)?

I need queue and status from the same row as the max datetime. ROW_NUMBER returns that full row. MAX alone returns only the timestamp.

Why trust daily over monthly for as-of?

Daily is the event log. Monthly is a convenience snapshot. When they disagree, I trust the log. I still used monthly as a sanity cross-check and as a fallback for the one account with no daily history before as-of.

Why the 23:59:59 fallback for account 37924?

It had monthly LEGAL in November but no daily row before the cut-off. I still want it in the as-of result, with a timestamp clearly before 2025-11-27, without pretending it was a real daily event.

What was the performance improvement?

Mostly avoid repeating DISTINCT discovery of qualifying accounts, select fewer columns, add indexes, ANALYZE, slightly larger cache. Verified with equal row counts and EXPLAIN plans.

11. Known limits

- Dataset is small, so wall-clock gain is small (~1.2x). The method still matters.
- Timings are median of warm runs in-process; I did not flush the OS page cache.
- Polars is only for ingest. All report logic is SQL on SQLite.

End of notes. Reading only this PDF should be enough to understand the full solution, the outputs, the numbers, and the reasoning.